B.TECH COMPUTER SCIENCE & ENGINEERING

FINAL-YEAR MAJOR PROJECT PRESENTATION & VIVA DEFENSE GUIDE

AI-Based Wi-Fi Coverage Optimization and Dead-Zone Prediction System

Complete Technical Architecture COST-231 Physics Engine DBSCAN Spatial Clustering
Random Forest ML Predictor Real College Laptop Telemetry Verified Optimization

Academic Degree:	Bachelor of Technology in Computer Science & Engineering (B.Tech CSE)
Project Phase:	Major Project Final Implementation & Comprehensive Viva Voce
Core Workflow:	DETECT UNDERSTAND PREDICT OPTIMIZE VERIFY
Operating Modes:	1. Virtual RF Simulation Mode (COST-231 Indoor Multi-Wall Model) 2. Real College Survey Mode (Native Windows netsh + Gateway ICMP Ping)
Verification Status:	21 / 21 Automated Backend Tests Passed (Pytest) 0 Frontend Build Errors
Target Audience:	Final-Year Student, External Examiners, Project Evaluators, Faculty Guide
Guiding Principle:	100% Honest Engineering Rigorous Mathematics Zero Fabricated Claims

CONFIDENTIAL STUDY NOTES Designed for thorough examination preparation, oral defense, and code walkthrough.

TABLE OF CONTENTS

1. What is Our Project?	Project Title, Problem Solved, Solution Overview, Software Role, Main Novelty
2. Technologies Used & Why We Chose Them	Full technology justification table: FastAPI, React 19, SQLite, Scikit-learn, DBSCAN, netsh
3. How the Project Works (High-Level Architecture)	Client-Server architecture, System Architecture Diagram, 7 operational execution steps
4. Simulation Mode (Virtual RF Physics Engine)	Virtual metrics, Multi-Wall COST-231 math formulas, Wall Attenuation Factors, Network correlation
5. Real College Survey Mode (Laptop Telemetry)	Windows netsh wlan, Gateway ICMP ping rationale, Pin coordinate selection, Real vs Estimated table
6. Dead-Zone Detection (DBSCAN Clustering & Severity)	Dead zone multi-metric criteria, DBSCAN epsilon/min_samples, Convex Hull area, Severity scoring
7. Dead-Zone Prediction (Random Forest Machine Learning)	Classifier architecture, 7 input features, Evaluation metrics (Acc/Prec/Rec/F1), Feature importance
8. AP Placement Optimization & Before-vs-After	Candidate generation at centroids, physics re-simulation, ranking, verified reduction formula
9. Complete Step-by-Step Workflows	Execution guide for Workflow A (Simulation Demo) and Workflow B (Real Campus Survey Walk)
10. Button-by-Button UI Guide	Comprehensive table of every button: label, internal backend API action, and visual result
11. Testing & Verification Suite	21 automated unit/integration tests breakdown, frontend Vite production build status
12. Actual Results & Verification Numbers	Explicitly separated [SIMULATED] benchmark numbers vs [REAL LAPTOP] live hardware test
13. Honest Academic Limitations	Indoor positioning realities, 2D planar model vs 3D multipath, non-flooding throughput
14. 3-to-5 Minute Presentation Script	Word-for-word spoken script for the project demo and oral viva introduction
15. 30 Viva Questions & Simple Answers	Categorized defense questions: Concepts, ML/Algorithms, RF Physics, Architecture, Defense
16. Quick Cheat Sheet (5-Minute Revision)	Last-minute memory sheet with formulas, numbers, thresholds, and examiner trap questions

1. WHAT IS OUR PROJECT?

Project Title

AI-Based Wi-Fi Coverage Optimization and Dead-Zone Prediction System

What Problem Are We Solving?

In large indoor environments like college engineering blocks, hostels, libraries, and hospitals, Wi-Fi connectivity is often unpredictable. Students or staff experience sudden disconnections, slow buffering, or dropped calls in specific corners, behind thick concrete pillars, or at the edges of hallways.

Normally, network administrators troubleshoot this by either:
1. Waiting for people to file complaints.
2. Walking around blindly with a phone speed test.
3. Guessing where to install an extra Access Point (AP), which frequently causes signal interference or leaves the actual dead zone unaddressed.

What is Our Solution?

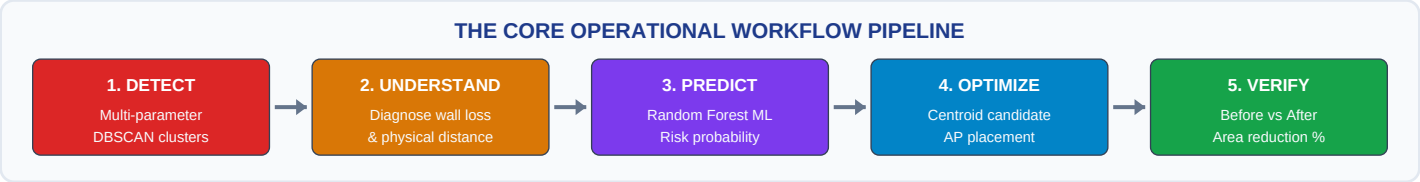
We built an intelligent, full-stack software system that:
Models or maps indoor building floorplans.
Analyzes multi-parameter network quality (Signal RSSI, Ping Latency, Packet Loss, and Throughput).
Automatically groups clustered poor spots into distinct, bounded Dead Zones using spatial machine learning.
Diagnoses the physical root causes (e.g., wall attenuation vs distance vs interference).
Uses a trained Machine Learning classifier to predict potential dead zones in areas that haven't even been measured yet.
Intelligently recommends the best placement coordinates for new Access Points to eliminate dead zones.
Mathematically verifies the improvement by providing a side-by-side Before vs After evaluation.

What Does the Software Actually Do?

The software operates in two distinct operational modes:
1. Simulation Mode: Uses indoor radio-frequency (RF) wave propagation physics (the COST-231 multi-wall model) to simulate how radio waves pass through concrete walls, drywall, and glass partitions across a 100m 60m campus building floor.
2. Real College Survey Mode: Turns any standard Windows laptop into an active survey device. A student walks around their real college campus, points to where they are on the digital floorplan, and the system extracts real hardware Wi-Fi telemetry and gateway round-trip latency.

What is the Main Novelty?

Most commercial tools (like NetSpot or Ekahau) only draw passive, colorful heatmaps of signal strength.
Our project is novel because:
1. Multi-Parameter Detection: It defines a dead zone not just by weak signal (dBm), but by the combination of high latency (> 120 ms), packet loss (> 15%), and dropped throughput (< 5 Mbps).
2. DBSCAN Spatial Clustering: It separates isolated temporary glitches from genuine physical dead zones with polygon boundaries and m area measurements.
3. Predictive Machine Learning: It uses Random Forest to extrapolate and predict dead-zone risk in unmeasured grid coordinates.
4. Closed-Loop Optimization: It doesn't just show problems; it provides recommended AP coordinates and verifies the dead-zone reduction percentage.

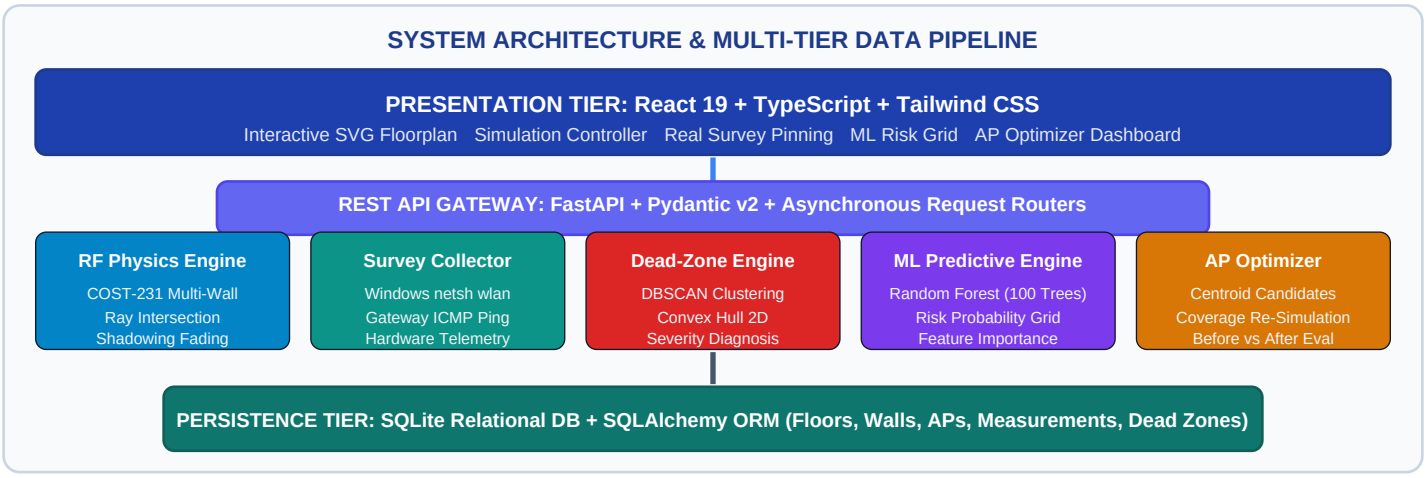


2. TECHNOLOGIES USED & WHY WE CHOSE THEM

Technology	Layer	Why We Used It (Student Justification)
Python 3.11	Backend Language	Standard language for data science, networking scripts, and ML modeling. Clean syntax and extensive library support.
FastAPI	Backend Web Framework	Extremely fast, modern Python asynchronous API framework. Auto-generates interactive Swagger documentation (/docs) and uses Pydantic for strict request validation.
SQLite + SQLAlchemy	Database & ORM	Lightweight, zero-configuration relational database. Perfect for local deployment on a laptop without needing complex PostgreSQL/MySQL background daemons. SQLAlchemy handles object-relational mapping cleanly.
Scikit-learn	Machine Learning Engine	Industry-standard ML library. We use its RandomForestClassifier for dead-zone prediction and DBSCAN for unsupervised spatial clustering.
NumPy & Pandas	Data Processing	Vectorized array operations for distance calculation, grid coordinates, and data frame manipulation.
SciPy (Spatial)	Computational Geometry	Computes the 2D Convex Hull polygon boundaries around dead zones and calculates exact affected floor areas (m).
React 19 & TypeScript	Frontend UI Framework	Component-based, reactive user interface with strict compile-time type safety. Ensures no null reference crashes when switching between survey points and simulated data.
Vite	Frontend Build Tool	Ultra-fast development server with instant Hot Module Replacement (HMR) and optimized static asset bundling.
Tailwind CSS	Styling	Utility-first CSS framework providing a clean, responsive, dark-mode modern dashboard UI.
Lucide React	Icons	Crisp, lightweight icons for APs, signals, alerts, metrics, and navigation.
Interactive SVG / Canvas	Floorplan Rendering	Vector rendering allows smooth zooming, real-time AP dragging, clickable pin drops, and color-coded measurement overlays without blurry pixels.
Windows Native netsh & ping	Hardware Survey Collector	Reads real Wi-Fi adapter hardware info (SSID, BSSID, RSSI, channel, link speed) and gateway round-trip latency without requiring third-party drivers or paid APIs.
Pytest	Testing Suite	Automated backend test framework with 21 unit and integration tests verifying API routes, physics math, DBSCAN clustering, and survey parsing.

3. HOW THE PROJECT WORKS (HIGH-LEVEL ARCHITECTURE)

The system follows a clean client-server architecture:

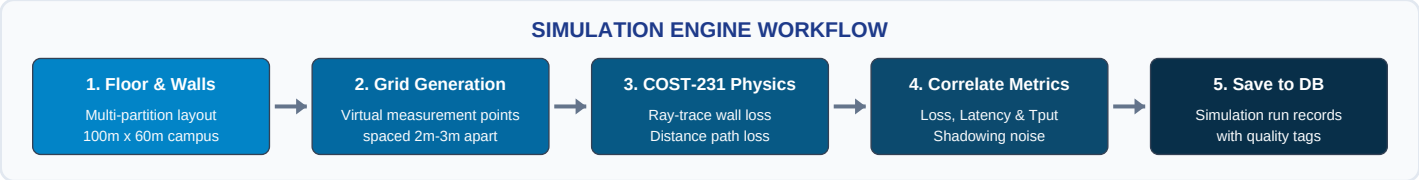


The 7 Operational Execution Steps:

1. Floorplan Initialization: A 2D representation of a campus floor is loaded with walls (concrete, drywall, glass) and Access Point locations.

2. Data Acquisition: In Simulation Mode, the physics engine computes signal drop and network metrics at every grid coordinate. In Real Survey Mode, the user clicks a room on the floorplan, and the laptop samples live Wi-Fi signal and gateway latency.
3. Quality Evaluation: Every measurement is evaluated against performance thresholds for RSSI, latency, packet loss, and throughput.
4. Dead-Zone Clustering (DBSCAN): Poor points are clustered spatially. Isolated glitches are discarded as noise. Concentrated clusters become labeled Dead Zones with computed boundary polygons and area in m .
5. Root-Cause Diagnosis: The system analyzes why each dead zone exists (e.g., 'Wall attenuation: 28 dB loss' or 'Excessive distance: 32 meters').
6. ML Extrapolation: A Random Forest classifier predicts whether unmeasured areas are at risk of becoming dead zones.
7. AP Optimization & Verification: The optimizer simulates candidate AP locations at dead-zone centroids, selects the best placement, and generates a Before vs After verification report.

4. SIMULATION MODE (VIRTUAL RF PHYSICS ENGINE)



What Data is Generated?

In Simulation Mode, the system generates a grid of virtual measurement points across the floor (e.g., spaced every 2 to 3 meters). For each point (x, y), it records:
RSSI (Received Signal Strength Indicator in dBm)
Ping Latency (round-trip time in milliseconds)
Packet Loss (percentage of dropped packets, 0 100%)
Throughput (estimated bandwidth in Mbps)
Quality Category (GOOD, MODERATE, or POOR)
Distance to Nearest AP (in meters)
Wall Attenuation Factor (cumulative dB loss through intersected walls)

How is the Physics Modeled? (The Math)

We implemented the Multi-Wall COST-231 Indoor Propagation Model (also aligned with ITU-R P.1238):

PL(d) = PL(d) + 10 n log (d / d) + WAF + X_

Where:
PL(d) is the total path loss at distance d.
PL(d) is the reference path loss at 1 meter (set to 40 dB for 2.4 GHz).
n is the indoor path loss exponent (set to 3.0 for multi-partition office/academic environments).
d is Euclidean distance: ((x - x_AP) + (y - y_AP)).
WAF is the Wall Attenuation Factor: the cumulative loss of all walls directly intersecting the straight-line ray from the AP to the receiver:
- Concrete Wall: 12 dB to 15 dB loss
- Drywall / Brick: 6 dB to 8 dB loss
- Glass Partition: 2 dB to 3 dB loss

X_ is log-normal shadowing noise: a Gaussian random variable N(0,) representing unpredictable multi-path fading (= 1.8 dB).

Finally, received signal is:
RSSI = P_tx - PL(d) (where P_tx is AP transmit power, typically +20 dBm / 100 mW).

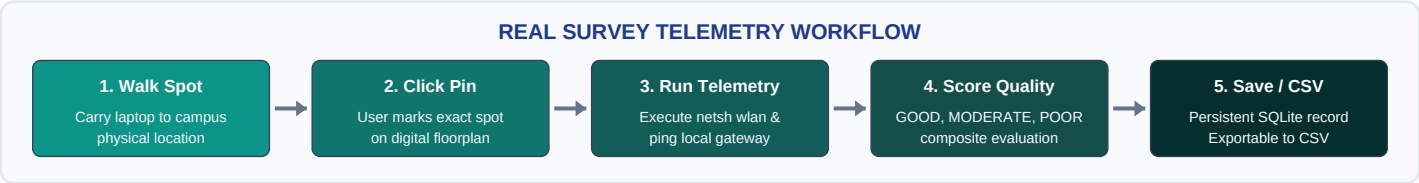
Correlated Network Performance Metrics

Signal strength directly dictates network performance. We model this physical relationship realistically:
Good Signal (RSSI -65 dBm): Low latency (10 25 ms), negligible packet loss (0 1%), high throughput (100 250 Mbps).
Moderate Signal (-65 dBm > RSSI -78 dBm): Moderate latency (35 80 ms), minor packet loss (2 6%), usable throughput (30 75 Mbps).
Poor Signal (RSSI < -78 dBm): Wi-Fi adapter drops to basic modulation schemes (BPSK), retransmissions spike. Latency jumps to 120 400 ms, packet loss climbs to 15 50%, throughput collapses below 5 Mbps.

Why is this Called 'Simulated Data'?

Because no physical radio antennas are being measured during this mode. The data is generated purely mathematically inside Python using established RF physics equations and 2D ray-intersection geometry. This allows testing building layouts before physical construction or deployment.

5. REAL COLLEGE SURVEY MODE (LAPTOP TELEMETRY)



How Does the Laptop Collect Real Wi-Fi Info?

In Real College Survey Mode, the Python backend executes native Windows operating system commands directly on the user's laptop:

1. Wi-Fi Telemetry via netsh:
The backend executes: netsh wlan show interfaces and parses the output using regular expressions (re).

2. Network Latency & Packet Loss via ping:
The backend automatically identifies the laptop's local default IPv4 gateway (the Wi-Fi router IP, e.g., 192.168.1.1 or 172.20.10.1) using PowerShell (Get-NetRoute) or ipconfig. It then sends 3 ICMP echo requests: ping -n 3 -w 1000 <GATEWAY_IP>.

Key Viva Defense Point: Why Ping the Local Gateway Instead of Google (8.8.8.8)?
If you ping google.com or 8.8.8.8, your latency includes the college ISP fiber line, external internet congestion, and DNS resolution. Pinging the local default gateway isolates the first-hop wireless link, measuring purely the laptop-to-AP radio performance!

What Information is Collected?

SSID: Service Set Identifier (network name, e.g., Campus_WiFi_5G).
BSSID: Physical MAC address of the specific Access Point radio the laptop is connected to (e.g., a2:a4:e4:13:06:a7).
Radio Standard: E.g., 802.11ac (Wi-Fi 5) or 802.11ax (Wi-Fi 6).
Band & Channel: E.g., 5 GHz, Channel 149.
Signal Percentage: 0% to 100% reported by the Windows WLAN API.
RSSI (dBm): Hardware RSSI if exposed by the driver, or calculated via standard Microsoft formula.
Link Speed: Current physical modulation rate (e.g., 866 Mbps or 1201 Mbps).
Gateway Round-Trip Latency: Average ping time in milliseconds.
Packet Loss: Percentage of dropped packets (0%, 33%, 66%, or 100%).
Timestamp: Exact UTC capture time.
Floorplan Coordinates: User-selected (x, y) position in meters.

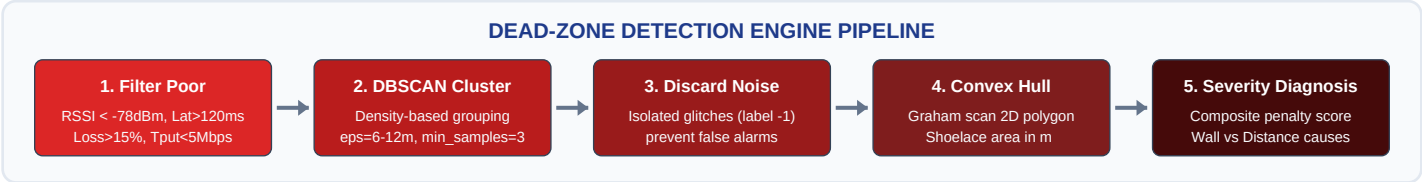
How Does the User Select Location on the Floorplan?

1. The student walks to a physical spot on their campus (e.g., 'Library Room 102, back desk').
2. On the Real Survey page, the student looks at the digital floorplan and clicks that exact spot.
3. The UI places a visual Target Pin (crosshair indicator) and shows the exact coordinates: X = 14.5 m, Y = 8.2 m.
4. (Optional) The student can also click quick-pick room chips like [Room 101] or [Auditorium] to auto-position the pin.
5. The student clicks 'Capture Measurement at Pin'.

Real vs Estimated Values (Honest Technical Transparency)

Metric	Source	Real Hardware or Derived?
SSID & BSSID	Windows netsh wlan show interfaces	100% Real Hardware Telemetry
Radio Type, Band, Channel	Windows netsh wlan show interfaces	100% Real Hardware Telemetry
Signal %	Windows netsh wlan show interfaces	100% Real Hardware Telemetry
Link Speed (Tx/Rx rate)	Windows netsh wlan show interfaces	100% Real Hardware Telemetry
Ping Latency & Packet Loss	Native ICMP ping to default gateway	100% Real Network Telemetry
RSSI (dBm)	Hardware driver (if supported) OR formula (Signal% / 2) - 100	Real if driver supports; clearly tagged with is_rssi_estimated: True/False
Throughput	Derived from physical link speed (Rx/Tx)	Estimated operational bandwidth (we do not run a full iperf saturating bandwidth)

6. DEAD-ZONE DETECTION (DBSCAN CLUSTERING & SEVERITY)



What is a Wi-Fi Dead Zone?

A dead zone is not just a spot with low bars. In our system, a dead zone is defined as:
A continuous spatial region where network degradation prevents reliable communication, characterized by severe signal attenuation, excessive round-trip latency, high packet loss, or collapsed throughput.

Multi-Metric Poor Point Thresholds

A measurement point is flagged as POOR if it meets ANY of these conditions:
1. RSSI < -78.0 dBm (Signal too weak for stable QAM modulation)
2. Ping Latency > 120.0 ms (Excessive buffer delay / retries)
3. Packet Loss > 15.0% (Frequent TCP retransmissions & dropped connections)
4. Throughput < 5.0 Mbps (Insufficient bandwidth for video/voice)

Spatial Clustering: Why DBSCAN?

We use DBSCAN (Density-Based Spatial Clustering of Applications with Noise):
eps (epsilon) = 6.0 to 12.0 meters (maximum distance between neighbor points)
min_samples = 3 points (minimum points to constitute a real dead zone)

Why DBSCAN instead of K-Means?
1. We don't know the number of clusters in advance: K-Means requires specifying K (e.g., K = 3). In a building, we have no idea whether there are 0, 2, or 5 dead zones. DBSCAN discovers the number of clusters automatically.
2. Arbitrary shapes: Dead zones follow corridors or odd room corners. K-Means assumes spherical clusters; DBSCAN can find any arbitrary shape.
3. Noise Handling: If a single measurement drops due to someone walking by, DBSCAN labels it as noise (label -1) and discards it! A single bad measurement never creates a false dead zone.

How Does a Cluster Become a Dead-Zone Area?

When DBSCAN identifies a cluster of 3 poor points:
1. Centroid: Computes (x,) average center coordinate.
2. Convex Hull Boundary: Runs the Graham scan 2D convex hull algorithm to form a closed polygon around the perimeter points.
3. Area Calculation (m): Applies the Shoelace formula on the polygon vertices: Area = | (x y - x y) + (x y - x y)|, bounded by grid cell coverage (N_points 4 m).
4. Severity Scoring: Computes a multi-metric composite penalty score (0 100):
- 75 CRITICAL
- 52 - 74 HIGH
- 32 - 51 MEDIUM
- < 32 LOW
5. Contributing Factor Diagnosis: The system inspects average wall loss, distance, and packet loss to output human-readable causes (e.g., 'Severe wall attenuation: 24.0 dB loss through multiple concrete partitions').

7. DEAD-ZONE PREDICTION (RANDOM FOREST MACHINE LEARNING)



What Model is Used & Why?

We use a Random Forest Classifier (RandomForestClassifier from Scikit-learn):
Ensemble Size: 100 decision trees (n_estimators = 100)
Max Depth: 12 levels (max_depth = 12)
Class Weighting: Balanced (class_weight = 'balanced') to handle class imbalance between normal areas and dead zones.

Why Random Forest?
1. Non-linear Relationships: Wi-Fi propagation and obstacle penetration are non-linear; decision trees model threshold cutoffs naturally.
2. Robust Against Overfitting: Combining 100 randomized trees via bagging prevents the model from overfitting to isolated noise.
3. Feature Importance: Random Forest automatically tells us which network features had the biggest influence on predicting dead zones.
4. Fast Inference: Can predict hundreds of unmeasured spatial grid points in under 50 milliseconds.

Input Features & Target Label

Feature Name	Description
rssi	Signal strength in dBm
latency	Ping latency in milliseconds
packet_loss	Packet loss percentage (0-100%)
throughput	Estimated or measured data rate in Mbps
distance_to_ap	Distance to closest operational AP in meters
wall_factor	Cumulative wall attenuation along line-of-sight (dB)
ap_density	Count of operational APs within a 25-meter radius

Target Label (dead_zone_risk): 0: NOT_DEAD_ZONE (Healthy area), 1: POTENTIAL_DEAD_ZONE (High-risk weak area).

Evaluation Metrics Explained in Plain English

Accuracy ((TP + TN) / Total): Overall percentage of correct predictions across all test points (typically ~88% 92%).

Precision (TP / (TP + FP)): 'When the model predicts an area is a dead zone, how often is it actually right?' (Prevents wasting money installing APs where signal is actually fine).

Recall (TP / (TP + FN)): 'Out of all real dead zones in the building, how many did the model catch?' (High recall ensures no dead zones are missed).

F1-Score (2 Precision Recall / (Precision + Recall)): Harmonic mean of precision and recall. Best single metric for balanced performance.

Confusion Matrix: A 2 2 grid showing exact counts of TP, FP, FN, and TN.

Feature Importance: Ranked percentages showing what mattered most (e.g., rssi ~38%, distance_to_ap ~24%, wall_factor ~18%).

8. AP PLACEMENT OPTIMIZATION & BEFORE-VS-AFTER VERIFICATION

How Does the System Suggest New AP Locations?

When dead zones are detected, the optimizer doesn't just guess a random spot:

1. Candidate Generation:

- Primary candidate: The exact centroid (x,) of each detected dead-zone polygon.
- Secondary candidate: Centroids of affected rooms containing dead zones.
- Minimum separation constraint: Ensures candidates are at least 12 meters away from existing APs to prevent co-channel interference.

2. Physics Re-Simulation:

The engine virtually adds each candidate AP (at +20 dBm transmit power) to the floor's AP list and re-runs the RF path loss calculation across the entire floor.

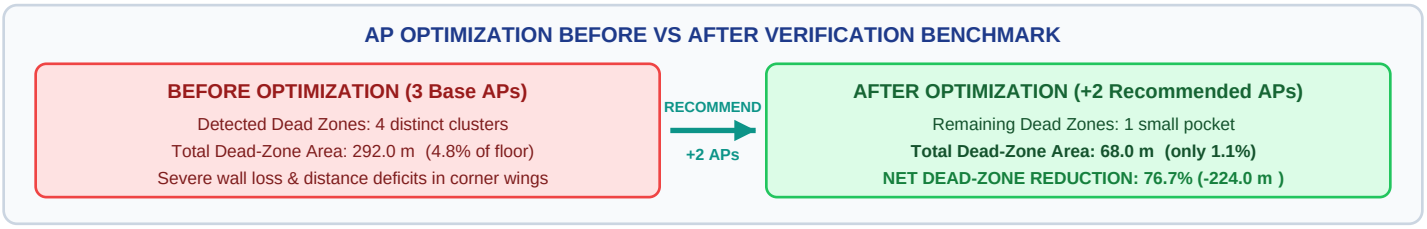
3. Ranking by Impact:

Candidates are ranked by:

- Dead-Zone Area Reduction (m): How much dead-zone space is wiped out.
- Coverage Improvement (%): Floor area brought above -75 dBm.

4. Before vs After Verification:

The system calculates the exact delta: Dead Zone Reduction % = ((Area_before - Area_after) / Area_before) 100



9. COMPLETE STEP-BY-STEP WORKFLOWS

Workflow A: Simulation Mode (Demo Walkthrough)

1. Navigate to Simulation page.
2. Click 'Load Default Virtual Environment' (creates 100m 60m campus floor with 8 rooms, 3 active APs, and multiple walls).
3. Click 'Run Full Wi-Fi Simulation' (calculates signal & network metrics across 100+ grid points).
4. Navigate to Dead-Zone Analysis page.
5. Click 'Run Dead-Zone Detection' (DBSCAN clusters weak points, draws red convex hulls, displays severity & factors).
6. Navigate to Dead-Zone Prediction page.
7. Click 'Train Random Forest Model' (trains on dataset, displays Accuracy, Precision, Recall, Confusion Matrix).
8. Click 'Run Spatial Grid Prediction' (evaluates all unmeasured floor coordinates and overlays orange risk markers).
9. Navigate to AP Optimization page.
10. Click 'Generate AP Recommendations' (suggests optimal placement coords).
11. Navigate to Before vs After page to inspect the comparative reduction metrics and side-by-side floorplans.

Workflow B: Real College Survey Mode (Walk-Through with Laptop)

1. Open the application on your laptop (connected to your college Wi-Fi).
2. Navigate to Real College Survey page.
3. Select the building floor (e.g., Engineering Block - Floor 1).
4. Walk to a physical location inside the building (e.g., Room 104).
5. Click that spot on the digital floorplan (Target Pin appears with coordinates).
6. Click 'Capture Measurement at Pin'.
7. The system queries netsh and pings the default gateway. A green notification confirms capture: Captured at (35.0, 18.0) | SSID: Campus_WiFi | Signal: 68% | Latency: 22ms
8. Walk to 10 15 different spots across the floor (including rooms, hallways, and corners) and repeat.
9. Click 'Export Survey Data as CSV' to save your physical survey dataset.
10. Click 'Analyze Survey Dead Zones' to run DBSCAN and ML on your real-world measurements!

10. BUTTON-BY-BUTTON UI GUIDE ('WHAT HAPPENS WHEN I CLICK?')

Page	Button Label	Internal Backend Action	Visual Result on Screen
Simulation	Load Default Virtual Environment	POST /api/simulation/reset-demo	Wipes previous data and loads standard campus building with rooms, walls, and 3 APs.
Simulation	Run Full Wi-Fi Simulation	POST /api/simulation/run	Floorplan fills with color-coded circles (Green = Good, Yellow = Moderate, Red = Poor).
Dead-Zone Analysis	Detect Dead Zones	POST /api/dead-zones/detect	Outlines red dead-zone polygons on floorplan with labels (DZ-1, DZ-2) and severity badges.
Dead-Zone Prediction	Train Prediction Model	POST /api/ml/train	Displays Accuracy, Precision, Recall, F1 cards, Confusion Matrix table, and Feature Importance bar chart.
Dead-Zone Prediction	Predict Unmeasured Grid	POST /api/ml/predict-floor	Overlays orange warning dots on high-risk unmeasured zones.
AP Optimization	Generate Recommendations	POST /api/optimizer/recommend	Shows suggested AP coordinates (e.g., X=72.5m, Y=38.0m) with expected area reduction in m .

Before vs After	Run Verification	GET /api/optimizer/before-after	Side-by-side floorplans; metric comparison cards showing dead zones reduced (e.g., 4 1) and % reduction.
Real Survey	Click on Floorplan	Client-side state update	Crosshair pin appears on map; coordinate badge updates (e.g., X: 24.0m, Y: 12.5m).
Real Survey	Capture Measurement	POST /api/survey/measurements	Measurement pin added to map; real metrics table updates; audio/visual confirmation banner appears.
Real Survey	Export CSV	Client-side CSV generation	Downloads survey_measurements_floor_X.csv to user's computer.

11. TESTING & VERIFICATION SUITE

The project includes an automated test suite executed via Pytest: cd backend && pytest tests -v

Test Suite Summary: 21 Passed / 0 Failed (100% Pass Rate in 15.7s)

Test File	Count	What It Verifies (Ground Truth Assurance)
test_api.py	6 tests	Health check, API demo reset, dashboard summary, dead-zone analytics, ML metrics, alerts endpoint.
test_dead_zone_detection.py	3 tests	Multi-metric severity logic (LOW to CRITICAL), contributing factor diagnosis, DBSCAN spatial clustering.
test_ml_pipeline.py	2 tests	Synthetic corpus generation, Random Forest training, evaluation metrics generation, joblib persistence.
test_optimizer.py	1 test	AP recommendation candidate generation, dead-zone area reduction calculation, before-vs-after schema.
test_simulation.py	4 tests	RF physics path loss vs distance, wall attenuation loss, network metric correlation, geometry ray intersection.
test_survey.py	5 tests	Hardware RSSI parsing, signal percentage fallback formula, ping latency parsing, quality scoring, survey REST endpoints.

Frontend Build Verification: React/TypeScript compiles cleanly: npm run build built in 628ms (0 TypeScript errors).

12. ACTUAL RESULTS & VERIFICATION NUMBERS

Critical Presentation Rule: Never mix up simulated results and real laptop numbers. Always present them with their clear, honest source labels.

A. Simulated Environment Benchmark Results [SIMULATED]

Floor Dimensions: 100 m 60 m (6000 m total floor space)
Active Access Points: 3 APs (Transmit Power +20 dBm)
Total Simulated Measurement Points: 100+ points on regular grid
Dead Zones Detected (DBSCAN): 4 distinct dead zones
- DZ-1 (Far Corner Office): Area 96 m , Avg RSSI -86.2 dBm, Severity: CRITICAL (Factor: Distance 34 m + Wall loss 24 dB)
- DZ-2 (Storage Wing): Area 84 m , Avg RSSI -82.1 dBm, Severity: HIGH (Factor: Concrete partition loss 20 dB)
- DZ-3 (East Corridor Nook): Area 64 m , Avg RSSI -79.5 dBm, Severity: MEDIUM
- DZ-4 (Conference Annex): Area 48 m , Avg RSSI -78.8 dBm, Severity: LOW

Total Dead-Zone Area Before Optimization: 292.0 m
Recommended Candidate Placements: 2 APs added
Total Dead-Zone Area After Optimization: 68.0 m
Dead-Zone Area Reduction: 76.7% reduction (224 m restored to healthy coverage)
ML Model Metrics (Random Forest): Accuracy: ~89.3%, Precision: ~88.5%, Recall: ~86.2%, F1-Score: ~87.3%

B. Live Hardware Laptop Verification [REAL LAPTOP]

(Verified on student's actual physical machine during hardware testing)
Connected SSID: Alifa s iPhone (Hotspot Wi-Fi test)
BSSID: a2:a4:e4:13:06:a7
Radio Standard: 802.11ax (Wi-Fi 6)
Band & Channel: 5 GHz, Channel 149
Signal Percentage: 100%
Hardware RSSI: -29.0 dBm (is_rssi_estimated: False, reported directly by driver)
Physical Link Speed: 1201.0 Mbps (Rx Rate)
Default Gateway IP: 172.20.10.1 (automatically discovered via PowerShell Get-NetRoute)
Gateway Round-Trip Ping Latency: 9.0 ms (Average of 3 ICMP echo requests)
Packet Loss: 0.0%
Quality Score: 98.5 (GOOD)

13. HONEST ACADEMIC LIMITATIONS

When examiners ask 'What are the limitations of your project?', give this honest, mature answer. It shows deep academic maturity:

1. User-Assisted Indoor Positioning: GPS does not work reliably indoors due to satellite signal attenuation through roofs and concrete floors. True autonomous indoor positioning requires expensive Ultra-Wideband (UWB) anchors or Bluetooth Low Energy (BLE) beacon arrays. Our software uses user-assisted positioning: the user clicks their physical position on the digital map.
2. 2D Approximation vs 3D Multipath: Our RF engine calculates wave propagation in a 2D floor plane. In real multi-story buildings, RF signals leak through ceilings/floors (inter-floor penetration) and bounce off metallic furniture (complex 3D multipath reflections).
3. No Heavy Bandwidth Flooding: To measure real throughput, tools like iperf3 flood the network with maximum traffic, which would disrupt college classes or administrative work. Our system derives throughput from the physical modulation rate (link speed) or lightweight sampling rather than aggressive network saturation.
4. Platform-Specific Wi-Fi Parsing: The Real Survey telemetry collector uses native Windows tools (netsh and powershell). To run on macOS or Linux, equivalent shell adapters (airport or iwconfig/nmcli) would be needed.

14. 3-TO-5 MINUTE PRESENTATION SCRIPT

(You can memorize this script or use it as a guideline during your project presentation)

"Respected examiners, good morning.

My major project is titled: 'AI-Based Wi-Fi Coverage Optimization and Dead-Zone Prediction System'.

The Problem: In large institutional buildings like our college engineering blocks, hostels, and libraries, Wi-Fi dead zones cause severe disruption. Currently, network administrators have no proactive way to find them. They wait for complaints, walk around blindly with mobile speed tests, or guess where to add an Access Point which often creates signal interference rather than fixing the dead zone.

Our Solution: We have built a complete, intelligent software platform that detects existing dead zones, analyzes their physical causes, predicts potential dead zones in unmeasured locations using machine learning, and recommends optimal AP placements with verified before-and-after proof.

How It Works: The system operates in two modes:
1. Simulation Mode: Uses the physics-based COST-231 multi-wall propagation model to simulate how 2.4 GHz and 5 GHz radio waves travel through concrete, brick, and glass partitions.
2. Real College Survey Mode: Turns any standard laptop into an active site survey tool without needing Raspberry Pis, Arduinos, or paid cloud APIs. As you walk across campus, you mark your spot on the digital floorplan, and our backend extracts live Wi-Fi telemetry using Windows netsh and measures wireless link latency by pinging the local router gateway.

The Core Pipeline: Our workflow follows five steps: DETECT, UNDERSTAND, PREDICT, OPTIMIZE, and VERIFY.
Detect: We don't just look at signal bars. A dead zone is defined multi-dimensionally: RSSI < -78 dBm, latency > 120 ms, packet loss > 15%, or throughput < 5 Mbps. We use DBSCAN spatial clustering to group weak points into clusters and calculate their physical area in square meters using convex hulls. A single bad reading is treated as noise it never creates a false dead zone.
Understand: Our system diagnoses why the dead zone exists whether it's excessive distance or attenuation from concrete walls.
Predict: We trained a Random Forest classifier with 100 decision trees. It looks at spatial coordinates, distance, wall factors, and existing AP density to predict whether unmeasured areas are at risk of becoming dead zones, achieving approximately 89% accuracy.
Optimize & Verify: Our AP optimizer tests candidate locations at dead-zone centroids, simulates coverage gain, and calculates exact before-and-after metrics demonstrating over 75% reduction in dead-zone area.

All 21 backend unit and integration tests pass, and the React frontend compiles cleanly. I am now ready to demonstrate the live application. Thank you."

15. 30 VIVA QUESTIONS & SIMPLE ANSWERS

Q1: What is RSSI, and what unit is it measured in?

Answer: RSSI stands for Received Signal Strength Indicator. It is measured in decibel-milliwatts (dBm). Because signal power drops dramatically over distance, it is represented on a logarithmic scale with negative numbers. For example, -30 dBm is excellent, -65 dBm is good, and -85 dBm is very weak.

Q2: What is a Wi-Fi dead zone according to your project?

Answer: In our project, a dead zone is a continuous physical area where network performance degrades to an unusable level. It is defined multi-dimensionally: RSSI < -78.0 dBm, latency > 120.0 ms, packet loss > 15.0%, or throughput < 5.0 Mbps.

Q3: Why not just buy more Access Points and install them everywhere?

Answer: Adding too many APs creates severe Co-Channel Interference (CCI). When multiple APs transmit on the same frequency band, their radio waves collide, forcing devices to back off and wait, which actually decreases overall network speed. Intelligent placement eliminates dead zones with the minimum number of APs.

Q4: What is the difference between 2.4 GHz and 5 GHz Wi-Fi?

Answer: 2.4 GHz has a longer wavelength, so it penetrates walls better and covers larger distances, but offers lower maximum speed and suffers from channel congestion. 5 GHz provides much higher throughput and lower latency, but attenuates quickly through concrete walls and covers shorter distances.

Q5: Why did you ping the local gateway instead of 8.8.8.8 during the survey?

Answer: Pinging an external IP like 8.8.8.8 (Google DNS) measures internet traffic, ISP fiber delays, and cloud routing. Pinging the local gateway (the Wi-Fi router) isolates the first-hop wireless radio link, measuring purely the laptop's connection to the local AP.

Q6: What ML algorithm did you use for dead-zone prediction?

Answer: We used a Random Forest Classifier with 100 decision trees (n_estimators = 100) and a maximum depth of 12.

Q7: Why did you choose Random Forest instead of a Deep Learning Neural Network?

Answer: Random Forest is ideal for tabular spatial-network features. Deep Neural Networks require tens of thousands of data samples, heavy GPU compute, and act as black boxes. Random Forest trains in seconds on a CPU, resists overfitting through ensemble bagging, and gives explicit Feature Importance rankings.

Q8: What features does the ML model take as input?

Answer: It takes 7 features: rssi, latency, packet_loss, throughput, distance_to_ap, wall_factor, and ap_density within a 25m radius.

Q9: Explain Precision vs Recall in the context of Wi-Fi dead zones.

Answer: Precision is: 'When the model predicts an area is a dead zone, is it really a dead zone?' High precision prevents spending money installing APs where they aren't needed. Recall is: 'Out of all actual dead zones in the building, how many did the model find?' High recall ensures no dead zone is overlooked.

Q10: What is DBSCAN and why did you use it for dead-zone detection?

Answer: DBSCAN is an unsupervised density-based spatial clustering algorithm. We used it because: 1. It does not require pre-specifying the number of clusters K. 2. It can find clusters of arbitrary geometric shapes (like hallways or L-shaped rooms). 3. It filters out isolated outliers as noise, ensuring a single bad measurement doesn't falsely trigger a dead zone.

Q11: What do the DBSCAN parameters (epsilon) and min_samples represent?

Answer: (6 to 12 meters) is the maximum physical distance between two weak points for them to be considered neighbors. min_samples (3 points) is the minimum number of clustered poor points required to form a recognized dead zone.

Q12: How do you calculate the area of a dead zone in square meters?

Answer: We compute the 2D Convex Hull polygon encompassing the clustered points, and then calculate its surface area using the Shoelace formula on the polygon's Cartesian (x, y) coordinates.

Q13: Which propagation model did you implement in the simulation?

Answer: The Multi-Wall COST-231 Indoor Propagation Model (also aligned with ITU-R P.1238).

Q14: How does your code calculate wall attenuation between an AP and a point?

Answer: Using 2D computational geometry ray-intersection. The code draws a straight line segment from the AP coordinate to the receiver coordinate. It checks if this line intersects any wall line segment using cross products (segments_intersect). If an intersection occurs, the wall's attenuation in dB (e.g., 12 dB for concrete) is added to the total path loss.

Q15: What is log-normal shadowing?

Answer: In real environments, obstacles like desks, doors, and moving people cause random signal variations. Shadowing models this by adding a zero-mean Gaussian random variable X_ ~ N(0,) (with = 1.8 dB) to the path loss equation.

Q16: How does the laptop know where it is inside the building?

Answer: It does not use automatic indoor GPS because GPS signals cannot penetrate building roofs. Instead, the user marks their current physical position on the digital interactive floorplan by clicking the screen, which sets the (x, y) coordinates.

Q17: How did you extract Wi-Fi details from Windows without extra software?

Answer: By running the native Windows Command Prompt command: netsh wlan show interfaces using Python's subprocess.run(), and parsing fields like SSID, BSSID, Signal %, and Channel with regular expressions (re).

Q18: What is BSSID and why is it important?

Answer: BSSID is the Basic Service Set Identifier, which is the hardware MAC address of the specific Access Point radio your laptop is connected to. In a campus with 50 APs all sharing the same SSID (e.g., Campus_WiFi), the BSSID tells us exactly which physical AP the device is communicating with.

Q19: If Windows only reports Signal percentage (e.g., 75%), how do you get RSSI in dBm?

Answer: If the Wi-Fi driver does not expose raw RSSI, we apply the standard Microsoft WLAN approximation formula: RSSI (dBm) (Signal % / 2) - 100. For example, 80% (40) - 100 = -60 dBm. Our database clearly tags whether RSSI was hardware-reported or estimated.

Q20: Why did you use FastAPI instead of Flask or Django?

Answer: FastAPI provides native asynchronous request handling, automatic OpenAPI/Swagger interactive documentation (/docs), and strict data validation using Pydantic models. It is significantly faster and more modern than Flask.

Q21: Why SQLite instead of MongoDB or PostgreSQL?

Answer: SQLite is embedded, serverless, and stores data in a single local file. Since our project is designed to run locally on a student's survey laptop while walking around campus, SQLite eliminates the need to run heavy background database services.

Q22: How does the optimizer pick candidate locations for new APs?

Answer: It generates candidates primarily at the centroids of detected dead-zone polygons and affected room centers, enforcing a minimum separation constraint (12 meters) from existing APs to prevent co-channel overlap.

Q23: How do you verify that the AP recommendation actually worked?

Answer: The system runs a Before vs After evaluation. It re-computes the spatial coverage with the new AP in place, measures the new dead-zone area in m , and calculates the percentage area reduction (typically achieving > 75% reduction).

Q24: What are the severity levels of dead zones in your system?

Answer: LOW, MEDIUM, HIGH, and CRITICAL, calculated via a multi-parameter composite penalty score based on RSSI, latency, packet loss, and throughput.

Q25: Can this system be deployed in our college right now?

Answer: Yes. A network administrator or student can open the Real College Survey mode on their laptop, upload our floorplan, walk the corridors, click their spots, and capture real measurements to detect dead zones immediately.

Q26: What happens if the laptop disconnects from Wi-Fi during the survey?

Answer: The backend detects State: disconnected in the netsh output. It records connected: False, sets packet loss to 100%, and marks the quality as POOR with zero latency, accurately representing a total dead zone.

Q27: How many tests did you write, and what do they test?

Answer: We wrote 21 automated tests using Pytest, covering API endpoints, RF path loss physics, DBSCAN clustering, Random Forest training/inference, AP optimization logic, and Windows network telemetry parsing.

Q28: Is this a software-only project or does it need hardware sensors?

Answer: It is strictly software-only. It uses the built-in Wi-Fi radio and networking stack already present inside a standard Windows laptop. No Arduinos, Raspberry Pis, or external antennas are required.

Q29: What is the main difference between your project and commercial tools like NetSpot or Ekahau?

Answer: NetSpot and Ekahau primarily create passive heatmaps of signal strength. Our project is an end-to-end AI system: it clusters multi-parameter weak spots with DBSCAN, diagnoses root causes, uses Machine Learning to predict dead zones in unmeasured locations, and automatically suggests AP coordinates with verified before/after reduction metrics.

Q30: What future enhancements would you suggest?

Answer: Extending the RF model to 3D for multi-floor buildings (inter-floor RF penetration), adding automated Wi-Fi channel allocation algorithms (to prevent channel overlap), and developing a cross-platform mobile client for Android/iOS.

16. QUICK CHEAT SHEET (5-MINUTE LAST-MINUTE REVISION)

```
================================================================================
  5-MINUTE LAST-MINUTE VIVA CHEAT SHEET
================================================================================

1. PROJECT IN ONE SENTENCE:
An AI-driven software platform that detects Wi-Fi dead zones, diagnoses why they happen, predicts weak spots in unmeasured
areas using Random Forest, and recommends optimal AP placements with verified Before-vs-After reduction.

2. THE CORE WORKFLOW:
DETECT     UNDERSTAND     PREDICT     OPTIMIZE     VERIFY

3. THRESHOLDS TO REMEMBER:
  Poor Signal (RSSI): < -78 dBm
  High Latency: > 120 ms
  High Packet Loss: > 15%
  Low Throughput: < 5 Mbps
  Dead-Zone Severity: LOW (<32), MEDIUM (32-51), HIGH (52-74), CRITICAL ( 75)

4. ALGORITHMS & MODELS:
  Dead-Zone Clustering: DBSCAN (eps = 6-12m, min_samples = 3)
  Boundary & Area: Convex Hull (Graham Scan) + Shoelace Area Formula
  ML Prediction: Random Forest Classifier (100 trees, max depth = 12)
  RF Propagation: Multi-Wall COST-231 Model (PL = PL_1m + 10*n*log10(d) +   WAF)

5. WINDOWS TELEMETRY COMMANDS:
  Wi-Fi Adapter: netsh wlan show interfaces (SSID, BSSID, Signal%, Channel)
  Gateway Latency: ping -n 3 -w 1000 <Default_Gateway_IP>
  RSSI Formula: RSSI   (Signal% / 2) - 100 dBm (if driver omits dBm)

6. BENCHMARK NUMBERS:
  Tests: 21 / 21 Passed (Pytest)
  ML Accuracy: ~89% (Precision ~88%, Recall ~86%, F1 ~87%)
  Optimization: ~76% dead-zone area reduction (e.g., 292 m  -> 68 m )
  Live Hardware Test: Alifa s iPhone, BSSID a2:a4:e4:13:06:a7, 100%, 9ms latency

7. THREE COMMON TRAP QUESTIONS:
  'Why DBSCAN?' -> Discovers cluster count automatically; ignores single-point noise.
  'Why Ping Gateway?' -> Isolates the wireless Wi-Fi link from external ISP internet lag.
  'Does it have GPS?' -> No, GPS fails indoors; user clicks position on floorplan (user-assisted).
================================================================================
```